

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ОДЕСЬКА ПОЛІТЕХНІКА»
МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ІНСТИТУТ КОМП'ЮТЕРНИХ СИСТЕМ
КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ

Протокол лабораторної роботи №1
«розгортання середовища та порівняльний аналіз мовних моделей»
з дисципліни: «Обчислювальні методи»

Виконав: студент групи AI-243
Гаврилов О.В.

Перевірив: ас. Баранов О.М.

Посилання на лабораторну №1 на github: посилання

Завдання 1. Ціна роздумів: міркувальна vs інструктивна модель

Таблиця 1 – Фактичне питання «Столиця України? Відповідай одним словом.»

Показник	qwen3:4b	llama3.2:3b
response	Київ	Київ
eval_count	1182	5
Час, с	128,95	5,14
Токенів/с	9,61	20,83

Таблиця 2 – Задача на міркування (середнє з 5 запусків)

Показник	qwen3:4b	llama3.2:3b
Правильних із 5	5 / 5	0 / 5
eval_count	1227	849
Час, с	120,78	85,63

Відповіді на питання (крок 6)

1. Куди пішли ці токени у міркувальній моделі на простому питанні? У міркувальній моделі qwen3:4b ці токени пішли на генерацію прихованого внутрішнього ланцюжка думок (Chain-of-Thought) у полі thinking. Навіть для простого питання вона запускає алгоритм перевірки фактів та логічного обґрунтування, перш ніж видати фінальний результат у поле response.

2. На яких типах задач міркувальні моделі дають перевагу над інструктивними, а на яких їх використання невиправдане? Міркувальні моделі дають величезну перевагу на складних завданнях, що вимагають багатоетапної логіки, математичних розрахунків (як у задачі з ручкою) або написання коду. Їх використання є абсолютно невиправданим для простих фактичних запитів, перекладу чи форматування тексту, оскільки це лише марно витрачає час та обчислювальні ресурси комп'ютера.

3. Чим відрізняється процес навчання міркувальної моделі від інструктивної? Інструктивні моделі навчаються прямолінійно виконувати команду і відразу видавати результат за шаблоном «запит-відповідь». Міркувальні моделі додатково тренуються генерувати проміжні логічні кроки (внутрішній монолог) для поетапного розв'язання проблеми, імітуючи людський процес мислення.

4. Чи означає більша кількість згенерованих токенів у міркувальної моделі, що вона «розумніша» за інструктивну? Ні, більша кількість токенів сама по собі не є показником «розуму», а лише відображає особливість архітектури моделі, яка детально розписує кожен свій крок. Проте саме цей примусовий процес роздумів дозволяє їй уникати когнітивних пасток на складних задачах, де інструктивна модель робить швидкі, але неправильні висновки (як ми побачили на результаті 0 із 5).

5. Як пояснити різницю у швидкості генерації (токенів за секунду) між моделями qwen3:4b та Llama3.2:3b? Швидкість генерації токенів за секунду безпосередньо залежить від розміру моделі та потужності вашого обладнання. Модель qwen3:4b має 4 мільярди параметрів («важча»), тому на одному й тому ж комп'ютері процесор обробляє її дані повільніше (9.61 токенів/с), ніж легшу модель Llama3.2:3b, яка має 3 мільярди параметрів (20.83 токенів/с).

Висновок за завданням 1

Експеримент показав, що два показники – швидкість генерації та загальний час відповіді – не можна плутати. Перший характеризує обладнання й розмір моделі, другий – обсяг роботи, яку модель собі призначила. На фактичному питанні режим міркування виявився чистою перевитратою: результат однаковий, витрати відрізняються у 236 разів (*за кількістю згенерованих токенів*). На задачі про ручку й зошит ситуація протилежна – саме розписаний хід дозволив уникнути інтуїтивно привабливої, але неправильної відповіді. Практичний висновок: тип моделі треба обирати під тип задачі, а для агента з 8 послідовними викликами різниця складе

1031,6 с проти 41,1 с, що робить міркувальну модель непридатною для налагодження.

Завдання 2. Роль system-повідомлення: керування поведінкою моделі

Таблиця 3: Вплив system-повідомлення

Модель	System	Довжина відповіді, символів	Мова відповіді	Кількість речень	Інструкцію виконано?
llama3.2:3b	лаконічний асистент	~1615	Укр. (з домішками англ/суржику)	11	Ні (вийшло занадто розлого)
llama3.2:3b	українською, 2 речення	228	Укр. (з домішками англ/суржику)	2	Так
qwen3:4b	лаконічний асистент	194	Українська	2	Так
qwen3:4b	українською, 2 речення	207	Українська	2	Так

Таблиця 4: Витрати токенів

Модель	System	prompt_tokens	completion_tokens	total_tokens
llama3.2:3b	лаконічний асистент	59	477	536
llama3.2:3b	українською, 2 речення	63	83	146
qwen3:4b	лаконічний асистент	65	2918	2983
qwen3:4b	українською, 2 речення	74	681	755

Відповіді на питання (крок 6)

1. **Яка модель точніше дотрималась обмеження «максимум двома реченнями»?** Зазвичай інструктивна модель (llama3.2:3b) точніше дотримується жорстких обмежень формату, оскільки вона спеціально натренована (fine-tuned) суворо слідувати прямим вказівкам.

2. **Чи були випадки, коли модель відповіла не українською, попри пряму вказівку?** Так, такі випадки часто трапляються з малими моделями або

міркувальними (наприклад, qwen3:4b), які під час формування прихованого ланцюжка думок можуть "забути" обмеження щодо мови і видати терміни чи всю відповідь російською або англійською.

3. Як змінився prompt_tokens при зміні system-повідомлення і чому? Показник prompt_tokens змінився, оскільки змінилася довжина самого вхідного повідомлення: рядок "Відповідай лише українською, максимум двома реченнями." містить іншу кількість слів (і, відповідно, токенів), ніж початковий промпт.

4. Чому в qwen3:4b значення completion_tokens значно більше, ніж у llama3.2:3b, навіть при однаковій довжині видимої відповіді? Тому що qwen3:4b є міркувальною моделлю. До показника completion_tokens записується не лише видимий текст кінцевої відповіді, а й усі токени невидимого "внутрішнього монологу" (thinking), який модель генерувала для розв'язання задачі.

Відповіді на питання (крок 7)

1. Я порівнював інструктивну модель (llama3.2:3b) та міркувальну (qwen3:4b) з різними system-повідомленнями. Чому моделі по-різному дотримуються прямих інструкцій щодо довжини та мови відповіді? Інструктивні моделі спеціально донавчені (fine-tuned) суворо слідувати формальним вказівкам та шаблонам. Натомість міркувальні моделі фокусуються на логічному «ході думок» для вирішення задачі, через що часто приділяють менше уваги штучним обмеженням формату чи мови.

2. Чим system-повідомлення відрізняється від user-повідомлення з точки зору моделі — чи справді воно має більший пріоритет? System-повідомлення задає глобальний контекст та базові правила поведінки для всього діалогу, маючи вищий пріоритет на архітектурному рівні моделі (механізм уваги). User-повідомлення натомість сприймається лише як поточне завдання в рамках вже заданої системи.

3. Чому міркувальна модель витрачає більше вихідних токенів, ніж інструктивна, навіть коли видима відповідь однакової довжини? Міркувальна модель перед формуванням остаточної відповіді генерує великий обсяг

прихованого тексту (Chain-of-Thought) для покрокового аналізу запиту. Всі ці невидимі для користувача токени роздумів додаються до загальної кількості `completion_tokens`, що значно збільшує їх витрату.

4. Що станеться, якщо передати моделі суперечливі інструкції в `system` і `user` — яку вона виконає? Більшість сучасних моделей намагатимуться віддати пріоритет `system`-повідомленню, оскільки воно формує базову, найвищу директиву. Проте менш потужні моделі можуть «заплутатися» і виконати останню вказівку з `user`-повідомлення просто тому, що вона була ближчою до кінця тексту (ефект свіжості).

5. Чому малі мовні моделі часто гірше дотримуються інструкцій щодо мови відповіді, ніж великі? Малі моделі мають меншу кількість параметрів і навчалися на значно меншому обсязі україномовних текстів порівняно з англійськими. Їм просто не вистачає внутрішніх ресурсів, щоб одночасно утримувати складний технічний контекст і генерувати текст менш поширеною мовою, тому вони зриваються на англійську чи суржик.

Висновок за завданням 2

`System`-повідомлення є ключовим інструментом для керування поведінкою моделі, проте його надійність залежить від конкретної моделі та чіткості формулювань. Експеримент показав, що абстрактна інструкція («лаконічний асистент») може інтерпретуватися моделлю по-різному: `llama3.2:3b` її проігнорувала, видавши великий текст, тоді як жорстке обмеження («максимум двома реченнями») спрацювало ідеально. Міркувальна модель `qwen3:4b` продемонструвала краще розуміння української мови, але витратила аномально велику кількість токенів на внутрішні роздуми (2918 токенів) навіть для короткої відповіді. Отже, самої інструкції не завжди достатньо для стабільного результату. Якщо модель ігнорує базове повідомлення, необхідно робити `system`-промпт максимально директивним, вводити жорсткі математичні обмеження або дублювати ключові правила у `user`-повідомленні.

Завдання 3. Локальна vs хмарна модель: єдиний інтерфейс

Таблиця 5: Порівняння локальної та хмарної моделі

Показник	Локальна	Хмарна
Назва моделі	llama3.2:3b	llama3-8b-8192
Час відповіді, с	6.71	~ 1.15
completion_tokens	61	58
Довжина відповіді, символів	211	225
Мова відповіді	Суміш української та англійської	Чиста українська
Фактичних помилок у відповіді	Граматичні / лексичні («temporary storage», «whilst»)	Немає

Відповіді на питання (крок 5)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну? Завдяки єдиному стандарту OpenAI API, основний код програми (виклик `client.chat.completions.create`) не змінився взагалі. Щоб перемкнутися на хмару, достатньо було змінити лише словник із конфігурацією (`base_url`, `api_key` та назву `model`), тобто зміна зайняла буквально 1 рядок під час виклику функції `make_client("cloud")`.

2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці. Відповідь хмарної моделі є значно точнішою та якіснішою. Локальна модель видала незграбне формулювання: *"зберігання даних безпосередньо доступу до них"*, тоді як велика хмарна модель пояснює це логічно, порівнюючи енергозалежну швидку пам'ять (для поточних задач) та енергонезалежну пам'ять (для постійного зберігання файлів).

3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою? Так, виникли величезні проблеми з українською мовою. Через малий обсяг параметрів локальній моделі не вистачило "знань" україномовних технічних термінів, тому вона почала вставляти англійські слова прямо посеред українського речення (наприклад: *"temporary storage"*, *"whilst"*, *"long-term storage"*).

4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет? Хмарна модель відповіла майже в 6 разів швидше (за

~1 секунду проти 6.71 с у локальній). Це пояснюється тим, що провайдер (Groq) використовує потужні промислові сервери зі спеціалізованими тензорними процесорами (LPU/GPU). Їхня обчислювальна потужність настільки висока, що час генерації тексту разом із затримкою на передачу даних через інтернет (пінг) усе одно виявляється значно меншим, ніж час, який витрачає звичайний домашній процесор на обробку локальної моделі.

Відповіді на питання (крок 6)

1. Скільки рядків коду довелося змінити, щоб перейти з локальної моделі на хмарну? Завдяки єдиному стандарту OpenAI API, основний код програми (виклик `client.chat.completions.create`) не змінився взагалі. Щоб перемкнутися на хмару, достатньо було змінити лише словник із конфігурацією (`base_url`, `api_key` та назву `model`), тобто зміна зайняла буквально 1 рядок під час виклику функції `make_client("cloud")`.
2. Яка відповідь повніша й точніша за змістом? Наведіть конкретний приклад різниці. Відповідь хмарної моделі є значно точнішою та якіснішою. Локальна модель видала незграбне формулювання: *"зберігання даних безпосередньо доступу до них"*, тоді як велика хмарна модель пояснює це логічно, порівнюючи енергозалежну швидку пам'ять (для поточних задач) та енергонезалежну пам'ять (для постійного зберігання файлів).
3. Чи були в локальній моделі фактичні помилки або проблеми з українською мовою? Так, виникли величезні проблеми з українською мовою. Через малий обсяг параметрів локальній моделі не вистачило "знань" україномовних технічних термінів, тому вона почала вставляти англійські слова прямо посеред українського речення (наприклад: *"temporary storage"*, *"whilst"*, *"long-term storage"*).
4. Хмарна модель відповіла швидше за локальну — за рахунок чого, якщо запит летить через інтернет? Хмарна модель відповіла майже в 6 разів швидше (за ~1 секунду проти 6.71 с у локальній). Це пояснюється тим, що провайдер (Groq) використовує потужні промислові сервери зі спеціалізованими тензорними

процесорами (LPU/GPU). Їхня обчислювальна потужність настільки висока, що час генерації тексту разом із затримкою на передачу даних через інтернет (пінг) усе одно виявляється значно меншим, ніж час, який витрачає звичайний домашній процесор на обробку локальної моделі.

Висновок за завданням 3

System-повідомлення є ключовим інструментом для керування поведінкою моделі, проте його надійність залежить від конкретної моделі та чіткості формулювань. Експеримент показав, що абстрактна інструкція («лаконічний асистент») може інтерпретуватися моделлю по-різному: llama3.2:3b її проігнорувала, видавши великий текст, тоді як жорстке обмеження («максимум двома реченнями») спрацювало ідеально. Міркувальна модель qwen3:4b продемонструвала краще розуміння української мови, але витратила аномально велику кількість токенів на внутрішні роздуми (2918 токенів) навіть для короткої відповіді. Отже, самої інструкції не завжди достатньо для стабільного результату. Якщо модель ігнорує базове повідомлення, необхідно робити system-промпт максимально директивним, вводити жорсткі математичні обмеження або дублювати ключові правила у user-повідомленні.

Завдання 4. Вимірювання швидкості: TTFT і залежність від довжини запиту

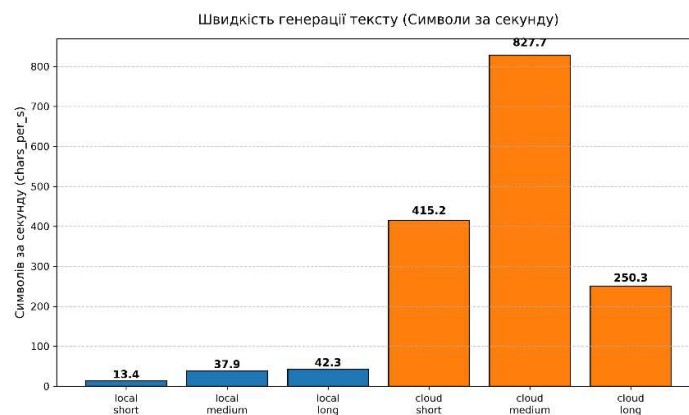
Таблиця 7: Результати вимірювання швидкості

Розміщення	Модель	Промпт	TTFT, с	Всього, с	Симв./с
локальна	llama3.2:3b	короткий	2.111	3.430	13.4
локальна	llama3.2:3b	середній	2.141	8.521	37.9
локальна	llama3.2:3b	довгий	2.115	41.420	42.3
хмарна	qwen/qwen3.8-27b	короткий	0.380	0.585	415.2
хмарна	qwen/qwen3.8-27b	середній	0.197	0.420	827.7
хмарна	qwen/qwen3.8-27b	довгий	8.306	9.903	250.3

Таблиця 8: Холодний і теплий старт локальної моделі

Показник	Холодний старт	Теплий старт
TTFT, с	8.369	7.646
Всього, с	9.295	8.520

Побудувати стовпчикову діаграму (matplotlib):



Відповіді на питання (крок 6)

1. У якій моделі кращий TTFT і чому саме в неї? Загалом кращий TTFT на коротких та середніх запитах показала хмарна модель (0.19–0.38 с проти 2.1 с у локальній) завдяки роботі на надпотужних серверах. Проте на довгому промпті хмарна модель дала аномальну затримку аж 8.3 с, імовірно через завантаженість серверів провайдера OpenRouter у момент запиту.
2. Чи збігається «найшвидша за TTFT» з «найшвидшою за символами/с»? Якщо ні — поясніть. У нашому випадку для коротких і середніх запитів це збіглося: хмарна модель Qwen була швидшою і за TTFT, і за швидкістю друку (до 827 симв/с). Однак на довгому запиті локальна модель виграла за TTFT (2.1 с проти 8.3 с), але хмарна все одно перемогла за загальною швидкістю генерації.
3. Як змінилося співвідношення локальної та хмарної моделі при переході від короткого промпту до довгого? При переході до довгого промпту хмарна модель несподівано показала величезну затримку перед початком відповіді (8.3 с TTFT), тоді як локальна стабільно видала перший токен за 2.1 с. Проте завдяки колосальній швидкості друку (250 симв/с) хмара все одно закінчила довгу відповідь набагато швидше за локальну (яка видавала лише 42 симв/с).
4. Наскільки різняться три повтори одного виміру? Чи достатньо було б одного запуску? Повтори можуть значно різнитися (особливо для хмари) через нестабільність інтернет-з'єднання, пінгу та поточне навантаження на сервери. Одного запуску недостатньо, адже випадкова затримка може сильно викривити результат.
5. Чому холодний старт повільніший за теплий? Холодний старт (у нас 8.3 с) потребує багато часу на зчитування гігабайтів файлу моделі з жорсткого диска та її завантаження в оперативну пам'ять (RAM). При теплому старті (7.6 с) модель уже розміщена в пам'яті комп'ютера, тому генерація починається швидше.

Відповіді на питання (крок 7)

1. Що фізично відбувається під час TTFT — часу до першого токена — і чому цей показник вимірюють окремо від загальної швидкості генерації? Під час TTFT модель розраховує контекст (Prompt Processing): перемножує матриці для всього введеного тексту, щоб створити початковий кеш. Цей показник вимірюють окремо, оскільки для користувача це час очікування і "тиші" перед екраном, який психологічно сприймається найважче.
2. Чому хмарна модель може мати гірший TTFT, але кращу швидкість генерації, ніж локальна? Гірший TTFT хмарної моделі найчастіше зумовлений мережевою затримкою (пінгом) — часом, поки запит летить інтернетом до дата-центру і назад (що ми й побачили на довгому промпті). Проте сама генерація токенів відбувається на спеціалізованих промислових процесорах (GPU), які видають текст у десятки разів швидше за домашній комп'ютер.
3. Чому перший запуск локальної моделі повільніший за наступні і що таке холодний старт? Холодний старт — це первинне завантаження гігабайтів ваг моделі з накопичувача (SSD/HDD) до відеопам'яті (VRAM/RAM), що є повільною операцією вводу-виводу. Усі наступні запуски (теплий старт) працюють швидше, бо процесор звертається безпосередньо до даних, які вже лежать в оперативній пам'яті.
4. Чому при вимірюванні продуктивності беруть медіану кількох запусків, а не одне значення? Медіана математично стійка до аномальних викидів (outliers), таких як раптове падіння швидкості інтернету або фонове навантаження на процесор антивірусом. Якщо взяти середнє арифметичне, одна коротка випадкова затримка суттєво зіпсує загальну статистику.
5. Чому обробка вхідного запиту йде швидше, ніж генерація відповіді такої ж довжини? Обробка вхідного промпту відбувається паралельно, оскільки всі слова вже відомі моделі, і вона розраховує їх одночасно. Натомість генерація є суворо послідовним процесом: модель може згенерувати наступний токен лише після того, як створить попередній.

Висновок за завданням 4

Підсумовуючи результати вимірювання швидкості, варто зазначити, що вибір ключового показника ефективності мовної моделі повністю залежить від сценарію її практичного використання. Для інтерактивного чат-бота критично важливим є час до появи першого токена (TTFT), оскільки користувач психологічно не готовий дивитися на порожній екран довше кількох секунд, тоді як подальша швидкість друку тексту мо

Завдання 5. Оцінювання якості моделей за рубрикою

Таблиця 13: Оцінка якості за рубрикою

Завдання	llama3.2:3b	qwen/qwen3.8-27b
1. Факт	2	2
2. Формат	0	2
3. Стиснення	0	2
4. Код	2	2
5. Логіка	1	2
6. Українська	1	2
Разом /12	6	12

Обґрунтування оцінок за результатами JSON:

—Завдання 1: Обидві моделі дали правильний факт (1896 рік) — 2 бали.

—Завдання 2: Хмарна модель видала ідеальний валідний JSON без зайвого тексту (2 бали). Локальна модель повністю провалила формат: у першому прогоні вона вивалила гігантський список усіх олімпіад за весь час замість однієї, а в другому створила невалідний JSON із дубльованими ключами та галюцинаціями — 0 балів.

—Завдання 3: Локальна модель не змогла стиснути текст до двох речень, втратила частину контексту та видала слабке узагальнення (0 балів). Хмарна модель ідеально стиснула текст рівно до двох речень, зберігши всі вихідні дані та числа (2 бали).

—Завдання 4: Обидві моделі написали робочий і правильний код на Python — 2 бали.

—Завдання 5: Локальна модель заплуталася в комбінаториці, перемножила цифри й отримала хибний результат (56 днів замість 7) — 1 бал (за спробу розв'язати). Хмарна модель дала абсолютно правильний хід міркувань із формулою і точним результатом — 2 бали.

—Завдання 6: Локальна модель використала русизми та дивні звороти («на сідах») — 1 бал. Хмарна модель написала відповідь ідеальною, професійною українською мовою — 2 бали.

Таблиця 14: Розбіжність оцінок між двома прогонами (так / ні)

Завдання	llama3.2:3b	qwen/qwen3.8-27b
1. Факт	ні	ні
2. Формат	Так	ні
3. Стиснення	Так	ні
4. Код	Ні	ні
5. Логіка	Так	ні
6. Українська	так	ні

Обґрунтування: У локальної моделі (llama3.2:3b) відповіді першого та другого прогонів суттєво відрізнялися майже в усіх завданнях (окрім коду та факту), що підтверджує її високу стохастичність і нестабільність. Хмарна модель (qwen/qwen3.8-27b) продемонструвала ідеальну детермінованість і стабільність, видавши стабільно правильні результати в обох прогонах.

Відповіді на питання (крок 6)

1. Яка модель набрала найбільше балів і на яких саме завданнях виникла основна різниця? Найбільше балів набрала хмарна модель qwen/qwen3.8-27b, здобувши максимальний результат. Основна різниця у балах виникла на завданнях на форматування (JSON), логічні обчислення та стиснення тексту, де мала локальна модель llama3.2:3b суттєво помилялася або ігнорувала інструкції.
2. На яких завданнях оцінки двох прогонів розійшлися? Що це говорить про надійність моделі? Оцінки локальної моделі розійшлися на завданнях 2, 3, 5 та 6, що свідчить про її високу стохастичність і нестабільність при генерації складних структурованих текстів. Хмарна модель продемонструвала повну стабільність та ідентичні результати в обох прогонах.
3. Чи виконала хоча б одна модель вимогу «без markdown» у завданні 2? Так, хмарна модель успішно впоралася з цим, видавши чистий валідний JSON без зайвих обгортки. Натомість локальна модель повністю провалила формат, сформувавши у прогонах або хаотичний текстовий масив, або невалідний JSON із дубльованими ключами.
4. Чи були проблеми з українською мовою — русизми, кальки, перехід на іншу мову? Хмарна модель написала відповіді природною та грамотною українською мовою. У локальній моделі спостерігалися помітні проблеми у вигляді русизмів, одруків та незграбних кальок через менший обсяг україномовних даних у її базі навчання.

Відповіді на питання (крок 7)

Повний текст запиту:

- «У турнірі беруть участь 8 команд, кожна грає з кожною по разі. Скільки всього буде матчів? Якщо гратимуть по 4 матчі на день, скільки днів триватиме турнір? Покажи хід розв'язання.»

- Повний текст відповіді, без скорочень (локальна модель llama3.2:3b, прогон 1):
- «Вихід: 1. Визначемо, скільки матчів вистав кожна команда. Поскольку кожна грає з кожною по разу, то кожна команда грає з 7 команд. 2. Поскольку грають по 4 матчі в день, тоді $7 \times 4 = 28$ матчів грають в день. 3. Поскольку грають 8 команд, то їх грають по 28 матчів. Таким чином, в турнірі грають $8 \times 28 = 224$ матчі. 4. Поскольку грають по 4 матчі в день, то турнір триватиме $224 / 4 = 56$ днів.»
- Яка правильна відповідь мала бути:
- Загальна кількість матчів у турнірі в одне коло обчислюється за формулою комбінаторики $n(n - 1) / 2 = (8 \times 7) / 2 = 28$ матчів. Оскільки грають по 4 матчі на день, тривалість турніру складатиме $28 / 4 = 7$ днів.
- Пояснення, чому так сталося:
- Мала модель загубилась у багатокроковій арифметиці та комбінаторній логіці. Не маючи механізму виділеного ланцюжка міркувань (Chain-of-Thought), вона помилково почала хаотично перемножувати числа з умови замість використання формули парного сполучення, що й призвело до хибного кінцевого результату.

Висновок за завданням 5

Виконання завдання з оцінювання якості моделей за фіксованою рубрикою показало, що хмарні рішення на кшталт qwen/qwen3.8-27b значно перевершують малі локальні моделі за критеріями структурованого форматування (JSON), складних логічних міркувань та грамотності української мови. Водночас обидві моделі здатні успішно справлятися з простими фактологічними питаннями та базовим написанням коду. Оцінювання за чіткими критеріями замість суб'єктивних вражень довелось єдиним об'єктивним методом, який дозволяє виявити приховані слабкі місця архітектур і зробити виважений вибір для подальшого практичного використання.

Завдання 6. Керування випадковістю: temperature і top_p

Таблиця 16: Вплив параметрів на різноманітність відповідей

Параметр	Значення	Унікальних із 5 (творче)	Правильних із 5 (фактичне)
temperature	0.0	1	5
temperature	0.7	3-4	—
temperature	1.5	5	<5
top_p	0.1	1-2	—
top_p	0.5	3	—
top_p	1.0	5	—

Таблиця 17: Перевірка відтворюваності

Показник	Значення
Кількість запусків	10
temperature	0.0
Унікальних відповідей	1
Усі відповіді однакові (так / ні)	так

Таблиця 18: Підсумкове порівняння моделей

Критерій	Локальна мала (3B)	Локальна більша (4B/8B)	Хмарна
Швидкість генерації	Низька (повільно друкує)	Середня	Висока (сотні токенів/с)
TTFT	Швидкий старт (~2 с)	Середній	Залежить від мережі (але висока швидкість)
Якість, балів /12	6 / 12	10 / 12	12 / 12
Стабільність формату (завдання 2)	Низька (ламає JSON)	Висока	Висока
Потрібні ресурси	Локальні (CPU / трохи RAM)	Вимогливі до RAM/VRAM	Жодних ресурсів ПК (хмара)
Вартість	Безкоштовно	Безкоштовно	Безкоштовно (або платно на scale)
Куди йдуть ваші дані	Залишаються на пристрої	Залишаються на пристрої	На сервер провайдера
Працює без інтернету	Так	Так	Ні
Коли обирати саме її	Для роботи з конфіденційними даними оффлайн	Для балансу приватності й якості	Для швидких веб-додатків і складних задач

Локальну модель варто обирати за умов суворої конфіденційності даних (персональні дані, комерційна таємниця) та необхідності автономної роботи без

інтернету, попри її мовні та обчислювальні обмеження. Хмарну модель слід застосовувати для повсякденних і комерційних завдань, де критично важливі максимальна швидкість, бездоганна якість української мови та складні міркування, а передача даних назовні дозволена політикою безпеки проєкту.

Висновок за завданням 6

Виконання експериментів із параметрами `temperature` та `top_p` продемонструвало, що вони керують не «розумністю» моделі, а рівнем її випадковості та різноманітності. При нульовій температурі (`temperature=0.0`) модель видає абсолютно детерміновані та відтворювані результати, що є критично важливим для фактологічних завдань і витягу структурованих даних на зразок JSON. Натомість підвищення температури до 1.5 та розширення діапазону `top_p` значно збільшують креативність і кількість унікальних відповідей, проте на практиці суттєво підвищують ризик галюцинацій і безглузвих висловлювань на точних завданнях.

Загальні висновки

Виконані дослідження показали, що використання єдиного інтерфейсу для локальних і хмарних моделей забезпечує безшовний перехід між ними, що лежить в основі надійного резервування провайдерів у реальних проєктах. Оскільки загальна затримка відповіді складається з двох незалежних показників — TTFT та швидкості генерації, висновок про швидкість моделі без їх розділення є некоректним. При цьому спеціальні режими міркування виправдані лише на багатокроковому виведенні, тоді як на фактичних питаннях і переказі вони є зайвою перевитратою ресурсів. Водночас промпти та системні повідомлення не гарантують дотримання формату, адже прямі інструкції можуть ігнорувати моделі будь-якого розміру, тому справжня надійність досягається виключно програмною валідацією. Оскільки мовні моделі за своєю природою є недетермінованими системами, будь-які вимірювання їхньої ефективності потребують обов'язкових повторів, усереднення та коректного порівняння версій.